



Athenaeum

A Library Management System

A complete, beginner-friendly guide to the project — what it does, how it is built, how every piece fits together, and how to run it. Written in plain language, from A to Z.

C++ Backend

React Frontend

REST API

JSON Storage

PROJECT DOCUMENTATION · Prepared for **Akshara Training**
Full-stack internship project

What's inside

1.	What Is This Project?	<i>the big picture</i>
2.	What Can It Do? (Features)	<i>every capability</i>
3.	The Two Halves: Frontend & Backend	<i>how a web app is split</i>
4.	How the Pieces Talk (The API)	<i>requests & responses</i>
5.	Where the Data Lives	<i>storage explained</i>
6.	Who Can Do What (Roles & Login)	<i>admin vs member</i>
7.	A Walk Through the Screens	<i>page by page</i>
8.	How Borrowing & Fines Work	<i>the core rules</i>
9.	The Full List of API Endpoints	<i>reference table</i>
10.	The Technology Used	<i>tools & libraries</i>
11.	How to Run It (Step by Step)	<i>setup guide</i>
12.	Project Folder Layout	<i>what each file is</i>
13.	Ideas to Extend It	<i>future work</i>
14.	Glossary of Terms	<i>plain-English definitions</i>

1 What Is This Project?

Athenaeum is a software system that helps a library keep track of its books, its members, and who has borrowed what. Think of it as a digital replacement for the old paper register and the cardboard "due date" slips tucked inside book covers.

A librarian (called an **admin**) can add books to the collection, register members, hand out books, take them back, and watch for overdue returns. A **member** can browse the catalogue and see the books they currently have on loan. Everything happens through a clean web page in the browser — no paper required.

IN ONE SENTENCE

It is a website where a librarian manages books and loans, and members can browse what's available — with the "brain" of the system written in the C++ programming language.

The word *Athenaeum* is an old name for a library or reading room, which is why the app is styled to look like a warm, wood-panelled library.

2 What Can It Do? (Features)

Here is everything the system can do, grouped by area:

Books (the catalogue)

- Add, edit, and delete books (admin only).
- Search instantly by title or author as you type.
- Filter by category, or show only books that are available.
- Track how many copies of each title exist and how many are free to borrow.

Members (the people)

- New people can register themselves as members.
- The admin can add, edit, or remove members from a dedicated panel.
- Each member has a role — either **admin** or ordinary **member**.

Circulation (borrowing & returning)

- Issue a book to a member for a 14-day loan period.
- Process returns; the system frees the copy back into the catalogue.
- Automatically calculate fines for late returns (₹2 per overdue day, configurable).
- Filter loans by active, returned, or all.

Dashboard & reports

- A live dashboard with totals: titles, copies, members, active loans, overdue count, and outstanding fines.
- A "Most Borrowed" leaderboard of the most popular titles.
- A "Collection Health" gauge showing what share of books are currently on the shelf.
- An overdue alert that appears when something needs attention.
- One-click export of the full loan history as a CSV file (opens in Excel).

3 The Two Halves: Frontend & Backend

Almost every modern web application is built from two cooperating parts. Understanding this split is the key to understanding the whole project.

FRONTEND — "THE FACE"

What the user sees and clicks. The buttons, tables, forms, colours and animations in the browser. Built with **React**. Its job is to show information nicely and to send the user's actions to the backend.

BACKEND — "THE BRAIN"

The logic and the rules. It decides who is allowed to do what, calculates fines, and remembers all the data. Built in **C++**. The user never sees it directly — it works behind the scenes.

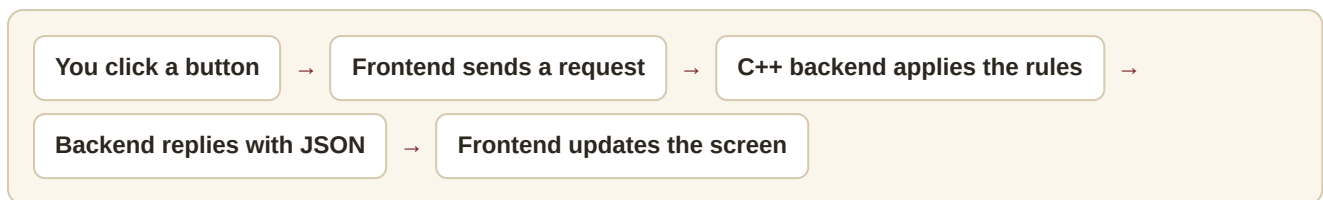
A SIMPLE ANALOGY

Think of a restaurant. The **frontend** is the dining room and the waiter — pleasant, visible, takes your order. The **backend** is the kitchen — hidden, but it does the real cooking. The waiter carries messages between you and the kitchen.

4 How the Pieces Talk (The API)

The frontend and backend are two separate programs. They communicate by sending messages over the network using a system called a **REST API**. You can think of the API as a menu of things the frontend is allowed to ask the backend to do.

When you click "Issue Book," the frontend sends a **request** to the backend saying "please lend book #3 to member #2." The backend checks the rules, updates its records, and sends back a **response** confirming success (or explaining the error). These messages travel in a simple text format called **JSON**.



Each request uses a **method** that signals intent — **GET** to read something, **POST** to create, **PUT** to update, and **DELETE** to remove. A full list of available requests appears in section 9.

5 Where the Data Lives

All the information — books, members, and loans — is saved in a single file named `library_db.json` on the computer running the backend. Every time something changes, the backend rewrites this file, so nothing is lost when the program stops.

This is deliberately simple. Many large systems use a full database (like MySQL or PostgreSQL), but for a project of this size a JSON file is easy to read, easy to back up, and needs no extra software to install.

GOOD TO KNOW

Login sessions (the proof that you are signed in) are kept only in memory, not in the file. So if the backend is restarted, everyone simply needs to sign in again — the books and loans remain safe.

6 Who Can Do What (Roles & Login)

When you sign in, the backend checks your email and password. If they match, it hands the frontend a secret **token** — a long random string that acts like a temporary wristband proving who you are. Every later request carries this token so the backend knows it's really you.

There are two roles, and the backend enforces the difference on every action:

ACTION	ADMIN	MEMBER
Browse and search the catalogue	Yes	Yes
See their own loans	Yes (all loans)	Yes (own only)
Add / edit / delete books	Yes	No
Manage members	Yes	No
Issue and return books	Yes	No
View dashboard & export reports	Yes	No

7 A Walk Through the Screens

Here is what a user actually sees, in the order they meet it.

The intro / landing screen

A welcoming page with the library's name in gold lettering and a hand-built 3D bookshelf. A single button, "Enter the Library," takes the visitor to the sign-in screen.

Sign in / register

Styled like a leather-bound book. Existing users sign in; new visitors can create a member account. Wrong details produce a clear, friendly error message.

Dashboard (admin)

The control centre. Cards show key totals at a glance, a leaderboard ranks the most-borrowed titles, a circular gauge shows collection health, and a red strip warns about overdue books.

Catalogue

The list of all books with a search bar and filters. Admins see extra "Edit" and "Delete" buttons and an "Add Book" button.

Members (admin)

A searchable table of everyone registered, each with a coloured initials badge. Admins can add a new member, edit an existing one, or remove one (with safety checks).

Circulation / My Loans

Admins see every loan and can issue new ones, process returns, and export a report. Members see a simpler version listing only their own borrowed books.

8 How Borrowing & Fines Work

The lending rules are the heart of any library system. They work like this:

1. An admin issues a book to a member. The system records today's date as the **issue date**.
2. It calculates a **due date** exactly 14 days later.
3. The number of available copies of that book drops by one.

4. When the book comes back, the admin processes the return. The copy becomes available again.
5. If the return happens after the due date, a fine is charged: **₹2 for every late day**.

WORKED EXAMPLE

A book is issued on the 1st and is due on the 15th. It is returned on the 20th — that is 5 days late. The fine is $5 \times ₹2 = ₹10$. While a book is still out and overdue, the dashboard shows the fine growing day by day so nothing is a surprise.

The loan period (14 days) and the fine rate (₹2/day) are not fixed forever — they are written as two simple settings at the top of the backend code and can be changed in one line.

9 The Full List of API Endpoints

This is the complete "menu" the backend offers. **Access** shows who is allowed to use each one. You do not need to memorise this — it is here for reference.

METHOD	ADDRESS	ACCESS	WHAT IT DOES
POST	/api/login	Public	Sign in, receive a token
POST	/api/register	Public	Create a new member account
GET	/api/books	Public	List, search, and filter books
GET	/api/categories	Public	List the distinct categories
POST	/api/books	Admin	Add a book
PUT	/api/books/:id	Admin	Update a book
DELETE	/api/books/:id	Admin	Delete a book
GET	/api/members	Admin	List all members
POST	/api/members	Admin	Add a member
PUT	/api/members/:id	Admin	Update a member
DELETE	/api/members/:id	Admin	Remove a member
GET	/api/loans	Logged in	List loans (members see only their own)
POST	/api/loans	Admin	Issue a book
POST	/api/loans/:id/return	Admin	Return a book and settle any fine
GET	/api/stats	Admin	Dashboard totals & leaderboard
GET	/api/report/loans.csv	Admin	Download loan history as CSV

10 The Technology Used

PART	TECHNOLOGY	WHY IT'S USED
Backend language	C++ (C++17)	Fast, powerful, and the focus of this project
Web server library	cpp-http lib	Lets C++ answer web requests; a single header file
JSON library	nlohmann/json	Reads and writes the JSON message format easily
Frontend library	React 18	Builds interactive screens from reusable pieces
Build tool	Vite	Runs and bundles the frontend quickly
Storage	JSON file	Simple, needs no separate database to install

WHY C++ FOR THE BACKEND?

Most tutorials use languages like JavaScript or Python for the server. Using C++ here is a deliberate learning choice: it shows that the same web-application ideas apply in any language, and it demonstrates handling web requests, JSON, and program logic in C++.

11 How to Run It (Step by Step)

You need **two** terminals open at the same time — one for the backend, one for the frontend. Do the backend first.

Step 1 — Start the C++ backend

You need a C++17 compiler (g++ on Linux/Windows-MinGW, or clang on Mac).

```
# move into the backend folder, then compile
cd backend
g++ -std=c++17 -O2 -pthread server.cpp -o server
# on Windows, add the sockets library:
# g++ -std=c++17 -O2 server.cpp -o server.exe -lws2_32

# now run it
./server          # Windows: server.exe
```

The backend now listens at `http://localhost:8080`. Leave it running.

Step 2 — Start the React frontend

You need Node.js version 18 or newer.

```
# in a second terminal, move into the frontend folder
cd frontend
npm install      # downloads the needed packages (first time only)
npm run dev      # starts the development server
```

Open the address it prints (usually `http://localhost:5173`) in your browser, and sign in with one of the demo logins. That's it.

COMMON FIRST-TIME ISSUES

- On Windows, if compiling fails with socket errors, make sure you added `-lws2_32`.
- If the page loads but nothing works, check that the backend terminal is still running.
- The very first `npm install` can take a minute — that's normal.

12 Project Folder Layout

The whole project lives in one folder. Here is what each item is:

```
library-system/
├── backend/
│   ├── server.cpp      ← the entire C++ backend (the brain)
│   ├── httpLib.h       ← web-server helper library
│   ├── json.hpp        ← JSON helper library
│   └── library_db.json ← the data file (created on first run)
└── frontend/
    ├── index.html      ← the page the browser opens first
    ├── package.json    ← lists the frontend's packages
    └── src/
        ├── App.jsx     ← all the screens and logic
        ├── styles.css  ← all the colours and 3D styling
        └── main.jsx     ← starts the React app
```

The two libraries (`httpLib.h` and `json.hpp`) are included directly so there is nothing extra to download for the backend — just compile and run.

13 Ideas to Extend It

If you want to take the project further, these are natural next steps:

- **Password security** — scramble (hash) passwords instead of storing them as plain text.
- **Reservations** — let members queue for a book that is currently out.
- **Email reminders** — notify members a day before a book is due.
- **Pagination** — show books in pages when the catalogue grows large.
- **PDF reports** — produce a printable report alongside the CSV export.
- **Real database** — swap the JSON file for MySQL or PostgreSQL as data grows.

14 Glossary of Terms

Frontend

The visible part of the app that runs in the browser — what you see and click.

Backend

The hidden part that holds the rules and the data. Here it is written in C++.

API

Application Programming Interface — the agreed set of requests the frontend can send to the backend.

REST

A common, tidy style of building an API using web addresses and methods like GET and POST.

Endpoint

One specific address in the API that does one job, e.g. `/api/books`.

JSON

A simple text format for sending structured data between programs. Easy for both people and computers to read.

Token

A temporary secret string that proves you are signed in, sent with each request.

Role

A label (admin or member) that decides what a user is allowed to do.

Compile

To turn human-readable C++ code into a program the computer can run.

Loan / Circulation

The act of lending a book to a member and later taking it back.

Fine

A small charge added when a book is returned after its due date.

Catalogue

The complete list of books the library owns.

IN SHORT

Athenaeum is a small but complete full-stack application: a friendly React screen on top, a C++ brain underneath, talking to each other over a simple web API, with all data kept in one tidy file. Everything in this guide describes that one idea from different angles.